

自然语言处理与大模型

笔记

整理人: Your Name

2026 年 1 月 19 日

目录

1 绪论 (2 学时宗成庆)	5
1.1 基本概念	5
1.2 问题挑战	5
1.3 技术方法	6
1.3.1 1. 理性主义 (Rationalism) - 符号逻辑	6
1.3.2 2. 经验主义 (Empiricism) - 统计学习	6
1.3.3 3. 连接主义 (Connectionism) - 深度学习	6
1.4 课程内容与考核	6
2 统计学习基础 (4 学时宗成庆)	7
2.1 概率论略览	7
2.1.1 基本概念回顾	7
2.1.2 随机过程 (Stochastic Process)	7
2.2 齐夫定律 (Zipf's Law)	7
2.3 信息论基础	8
2.3.1 熵 (Entropy) 与语言熵	8
2.3.2 互信息 (Mutual Information)	8
2.3.3 相对熵与交叉熵 (重点辨析)	8
2.3.4 困惑度 (Perplexity)	8
2.4 统计学习模型分类	9
2.4.1 生成式模型 (Generative Model)	9
2.4.2 判别式模型 (Discriminative Model)	9
2.5 最大熵模型 (MaxEnt)	9
2.5.1 核心原理	9
2.6 在 NLP 的应用: 词义消解	9
2.6.1 数学定义	9
2.6.2 3. 应用实例: 词义消歧 (WSD)	10
2.6.3 4. 参数训练: 通用迭代尺度法 (GIS)	10

2.7	条件随机场 (CRF)	11
2.7.1	为什么需要 CRF?	11
2.7.2	线性链 CRF 模型	12
2.7.3	CRF 的解码: Viterbi 算法	12
3	N 元文法模型 (6 学时宗成庆)	14
3.1	模型定义 (Model Definition)	14
3.1.1	1. 语言模型与链式法则	14
3.1.2	2. 马尔可夫假设 (Markov Assumption)	14
3.1.3	3. 常见 N-gram 模型	14
3.2	参数估计 (Parameter Estimation)	15
3.2.1	最大似然估计 (MLE)	15
3.2.2	实例计算 (The "John read Moby Dick" Example)	15
3.3	数据平滑 (Data Smoothing)	15
3.3.1	为什么要平滑?	15
3.3.2	1. 加 1 平滑 (Add-one / Laplace Smoothing)	16
3.3.3	2. Good-Turing 估计 (重点理解)	16
3.3.4	3. 线性插值法 (Linear Interpolation)	16
3.4	模型评价	17
3.5	典型应用: 音字转换 (Pinyin-to-Character)	17
4	传统 NLP 关键技术 (6 学时宗成庆)	18
4.1	词语切分与命名实体识别 (Lexical Analysis)	18
4.1.1	1. 分词难点: 歧义与未登录词	18
4.1.2	2. 机械切分算法 (Rule-based)	18
4.1.3	3. 统计分词方法 (Statistical)	19
4.1.4	4. 子词切分 (Subword Segmentation)	19
4.1.5	5. 词性标注与命名实体识别	19
4.2	句法分析 (Syntactic Parsing)	19
4.2.1	1. 短语结构分析 (Phrase Structure Parsing)	19
4.2.2	2. 依存句法分析 (Dependency Parsing)	20
4.2.3	3. 附录: 汉英句法结构对比 (重点)	20
4.3	语义分析 (Semantic Analysis)	20
4.3.1	1. 语义歧义 (Semantic Ambiguity)	20
4.3.2	2. 语义网络 (Semantic Network)	21
4.3.3	3. 语义角色标注 (SRL)	21
5	神经网络与语言模型 (6 学时张家俊)	22
5.1	1. 神经网络基础 (Neural Networks Overview)	22
5.1.1	1.1 神经元模型 (M-P Model)	22
5.1.2	1.2 核心激活函数	22

5.2	2. 前馈神经网络语言模型 (FFNN-LM)	22
5.2.1	2.1 动机: 解决 N-gram 的两大痛点	22
5.2.2	2.2 模型架构 (Bengio et al., 2003)	22
5.2.3	2.3 反向传播推导 (Back-Propagation Details)	23
5.3	3. 循环神经网络语言模型 (RNN-LM)	23
5.3.1	3.1 核心公式	23
5.3.2	3.2 BPTT 与梯度问题	23
5.4	4. 长短时记忆网络 (LSTM)	23
5.4.1	4.1 核心机制: 三门一胞	23
5.5	5. 自注意力机制 (Self-Attention)	24
5.5.1	5.1 动机	24
5.5.2	5.2 计算步骤 (Scaled Dot-Product Attention)	24
5.5.3	5.3 多头注意力 (Multi-Head Attention)	25
5.5.4	5.4 GPT (Generative Pre-trained Transformer)	25
6	文本表示 (Text Representation) (3 学时张家俊)	25
6.1	1. 向量空间模型 (Vector Space Model, VSM)	25
6.1.1	1.1 基本概念	25
6.1.2	1.2 长度规范化	26
6.1.3	1.3 离散符号表示的缺陷	26
6.2	2. 词语的表示学习 (Word Embedding)	26
6.2.1	2.1 分布式假设 (Distributional Hypothesis)	26
6.2.2	2.2 基于语言模型的方法	26
6.2.3	2.3 C&W 模型 (Collobert & Weston, 2008)	26
6.2.4	2.4 Word2Vec (Mikolov et al., 2013) [核心考点]	26
6.2.5	2.5 GloVe (Global Vectors)	27
6.3	3. 短语与句子表示学习	27
6.3.1	3.1 基础组合方法	27
6.3.2	3.2 基于神经网络的方法	28
6.4	4. 文档表示学习	28
6.4.1	4.1 层次化注意力模型 (HAN)	28
6.4.2	4.2 Doc2Vec (Paragraph Vector)	28
7	大语言模型及其训练 (6 学时张家俊)	29
7.1	1. 预训练语言模型演进	29
7.1.1	1.1 ELMo (Embeddings from Language Models)	29
7.1.2	1.2 GPT (Generative Pre-Training)	29
7.1.3	1.3 BERT (Bidirectional Encoder Representations from Transformers)	30
7.2	2. 训练数据工程	30
7.2.1	2.1 数据来源 (Sources)	30

7.2.2	2.2 数据处理流水线 (Pipeline)	30
7.2.3	2.3 数据配比 (Data Mixture)	31
7.3	3. 高效并行训练方法	31
7.3.1	3.1 显存占用分析	31
7.3.2	3.2 3D 并行策略 (3D Parallelism)	31
7.3.3	3.3 显存优化技术	32
7.4	4. 微调与对齐	32
7.4.1	4.1 指令微调 (Instruction Tuning)	32
7.4.2	4.2 基于人类反馈的强化学习 (RLHF)	32
7.4.3	4.3 直接偏好优化 (DPO, Direct Preference Optimization)	33
7.5	5. 提示学习	33
7.5.1	5.1 上下文学习 (In-Context Learning, ICL)	33
7.5.2	5.2 思维链 (Chain-of-Thought, CoT)	33
8	大模型其他知识 (6 学时张家俊)	34
8.1	1. 多语言大模型 (Multilingual LLM)	34
8.1.1	1.1 背景与挑战	34
8.1.2	1.2 关键技术	34
8.2	2. 多模态大模型 (Multimodal LLM)	34
8.2.1	2.1 架构范式	34
8.2.2	2.2 经典模型	35
8.3	3. 检索增强生成 (RAG)	35
8.3.1	3.1 核心动机	35
8.3.2	3.2 RAG 进化范式	35
8.3.3	3.3 关键技术细节	35
8.3.4	3.4 评估体系 (RAGAS)	36
9	NLP 应用任务 (6 学时张家俊)	37
9.1	1. 文本分类 (Text Classification)	37
9.1.1	1.1 任务定义	37
9.1.2	1.2 传统机器学习方法	37
9.1.3	1.3 深度学习方法	37
9.2	2. 文本聚类 (Text Clustering)	37
9.3	3. 信息抽取 (Information Extraction)	38
9.3.1	3.1 命名实体识别 (NER)	38
9.3.2	3.2 关系抽取 (Relation Extraction)	38
9.3.3	3.3 事件抽取 (Event Extraction)	38
9.4	4. 机器翻译 (Machine Translation)	39

10 课程总结与展望 (3 学时宗成庆)	39
10.1 课程内容回顾	39
10.2 学科现状分析	39
10.3 未来展望	39
10.4 关于课程考核	39
11 考核 (2 学时宗成庆)	40
11.1 考试	40
附录：常用 LaTeX 模板	41

1 绪论 (2 学时宗成庆)

1.1 基本概念

- 语言与自然语言：语言是用于表达意思、交流思想的工具，也是一个抽象的数学系统。自然语言是指人类社会发展中自然产生的语言。
- 学科术语辨析：

NLU (自然语言理解) 侧重探索人类语言认知过程，模仿人类思维，属于 AI 早期核心问题 (1956s)。

CL (计算语言学) 侧重语言学角度，通过建立形式化计算模型来分析语言，偏基础理论 (1960s)。

NLP (自然语言处理) 侧重工程与技术实现，利用计算机对文本进行加工处理 (1970-80s)。

HLT (人类语言技术) 范围更广，涵盖语音、文本等多模态技术 (1980s)。
- 重要历史节点：
 - 1954 年：Georgetown 大学与 IBM 用 IBM-701 实现了世界上第一个机器翻译系统。
 - 1956 年：达特茅斯会议，人工智能 (AI) 诞生，NLU 成为 AI 核心问题之一。
 - 1966 年：美国科学院发布 ALPAC 报告，认为机器翻译昂贵且质量低，直接导致 NLP 研究进入长达十年的低谷期。

1.2 问题挑战

自然语言处理的核心困难在于歧义性 (Ambiguity) 和不确定性。

1. 歧义现象：

- 分词/结构歧义：例如“门把手弄坏了”可切分为“门/把手/弄坏了”或“门/把/手/弄坏了”。
- 语义歧义：例如“喜欢乡下的孩子”(是喜欢“乡下”这个地方，还是喜欢那里的“孩子”?)。

2. 未知语言现象：新词（如“内卷”、“给力”）、新含义（“潜水”）、新用法的不规范性。
3. 知识获取困难：常识往往是隐蔽的，且不同语言间存在概念差异（Culture Gap）。

1.3 技术方法

NLP 技术发展主要经历了三个范式的演变：

1.3.1 1. 理性主义 (Rationalism) - 符号逻辑

- 核心思想：基于规则 (Rule-based)，通过人类专家编写规则和词典，进行“分析-转换-生成”。
- 代表人物：Chomsky (句法结构)。
- 优缺点：可解释性强，但覆盖率低，鲁棒性差，难以处理大规模真实文本，跨语言移植难。

1.3.2 2. 经验主义 (Empiricism) - 统计学习

- 核心思想：基于统计 (Statistical)，利用大规模真实语料，统计语言规律的可能性（概率）。
- 代表模型：HMM, SVM, CRF, n-Gram。
- 核心公式 (SMT 噪声信道模型)：

$$\hat{T} = \arg \max_T P(T|S) = \arg \max_T \frac{P(T) \times P(S|T)}{P(S)} \approx \arg \max_T P(T) \times P(S|T)$$

其中 $P(T)$ 为语言模型 (Language Model)，保证译文通顺； $P(S|T)$ 为翻译模型 (Translation Model)，保证语义对应。

- 优缺点：不需要深层句法分析，开发周期短；但难以处理长距离依赖，对语料规模和质量依赖大。

1.3.3 3. 连接主义 (Connectionism) - 深度学习

- 核心思想：基于神经网络，使用连续向量 (Embedding) 表示语言单位，端到端 (End-to-End) 训练。
- 代表模型：CNN, RNN, LSTM, Transformer, BERT, GPT 系列。
- 演变路线：SMT (1989) → NMT (2013) → ChatGPT (2022)。
- 优缺点：性能卓越，无需繁琐的人工特征工程；但模型如同“黑盒”可解释性差，且对算力和数据要求极高。

1.4 课程内容与考核

- **课程结构：**基本概念 → 基础理论 (统计/N-gram) → 传统技术 (分词/句法/语义) → 深度学习与 LLMs (Transformer/BERT/GPT/多模态) → 应用系统。
- **考核方式：**闭卷考试 (60%) + 课程实践 (40%)。

2 统计学习基础 (4 学时宗成庆)

复习重点: 本章复习重点

1. 概率论与随机过程: 掌握语言的稳态遍历性假设。
2. 齐夫定律 (Zipf's Law): 理解长尾效应及其对 NLP 数据稀疏的影响。
3. 信息论核心指标: 熵、联合熵、条件熵、互信息、KL 散度、交叉熵、困惑度。
4. 统计学习分类: 生成式模型 (HMM) vs 判别式模型 (CRF, SVM)。
5. 最大熵模型 (MaxEnt): 原理、约束条件、参数估计算法 (GIS, IIS, L-BFGS)。
6. 条件随机场 (CRF): 解决标注偏置问题、全局归一化、Viterbi 解码。

2.1 概率论略览

2.1.1 基本概念回顾

- 贝叶斯法则 (Bayes' Theorem): NLP 中常用于求解逆向概率 (如拼写纠错、机器翻译)。

$$P(Y|X) = \frac{P(X|Y)P(Y)}{P(X)} \propto P(X|Y)P(Y)$$

其中 $P(Y)$ 为先验概率 (Language Model), $P(X|Y)$ 为似然概率 (Observation Model)。

- 最大似然估计 (MLE):

$$\hat{\theta} = \arg \max_{\theta} P(D|\theta) = \arg \max_{\theta} \sum_{i=1}^N \ln P(x_i|\theta)$$

2.1.2 随机过程 (Stochastic Process)

语言被视为一个随机过程 $\{\xi_t, t \in T\}$ 。为了使语言的统计规律可被研究, NLP 引入了两个重要假设:

1. 稳态性 (Stationarity): 语言的统计特性 (如词频、共现概率) 不随时间推移而改变。即 $P(w_t) \approx P(w_{t+k})$ 。
2. 遍历性 (Ergodicity): 单个样本在长时间内的统计特性等于所有样本在同一时刻的统计特性。这意味着我们可以通过在大规模语料库 (空间) 上的统计来近似语言 (时间) 的概率分布。

2.2 齐夫定律 (Zipf's Law)

- 定义: 在自然语言语料库中, 一个单词出现的频率 f 与其排名 r 成反比。

$$f \times r = C \quad (\text{常数})$$

或者写成对数形式, $\log(f)$ 与 $\log(r)$ 呈线性关系: $\log f = \log C - \log r$ 。

- **长尾效应 (Long Tail Effect):**

- **高频词:** 极少数词 (如 "the", "的", "是") 占据了极高的频率。
- **低频词 (长尾):** 绝大多数词出现的频率非常低。
- **NLP 挑战:** 无论语料库多大, 总会遇到未登录词 (Unknown Words/OOV) 和数据稀疏 (Data Sparsity) 问题。

2.3 信息论基础

2.3.1 熵 (Entropy) 与语言熵

衡量随机变量的不确定性。

$$H(X) = - \sum_x P(x) \log_2 P(x)$$

- **常识数据:** 英文字母的熵约为 4.03 bits, 汉字的熵约为 9.71 bits (冯志伟, 1989)。

2.3.2 互信息 (Mutual Information)

- **定义:** $I(X; Y) = H(X) - H(X|Y)$ 。表示知道 Y 后 X 不确定性的减少量。
- **点式互信息 (PMI):** 用于衡量两个具体事件 (如两个词) 的相关性。

$$PMI(x, y) = \log_2 \frac{P(x, y)}{P(x)P(y)}$$

- **应用:** 在汉语分词中, 如果两个字 x, y 的 PMI 值很高, 说明它们结合紧密, 倾向于成词; 反之则可能需要断开。

2.3.3 相对熵与交叉熵 (重点辨析)

- **相对熵 (KL Divergence):** 衡量两个分布 P (真实) 和 Q (模型) 的距离。

$$D_{KL}(P||Q) = \sum_x P(x) \log \frac{P(x)}{Q(x)}$$

- **交叉熵 (Cross Entropy):**

$$H(P, Q) = H(P) + D_{KL}(P||Q) = - \sum_x P(x) \log Q(x)$$

- **关系与作用:** 在机器学习中, 最小化交叉熵等价于最小化相对熵。因为真实分布 $P(x)$ 是固定的, 其熵 $H(P)$ 是常数。我们通过最小化交叉熵损失函数, 使模型分布 $Q(x)$ 逼近真实分布 $P(x)$ 。

2.3.4 困惑度 (Perplexity)

语言模型评价指标。 $PP_q = 2^{H(L, q)} \approx 2^{-\frac{1}{n} \log q(x_1^n)} = [q(x_1^n)]^{-\frac{1}{n}}$ 。困惑度越小, 模型越好。其中 $H(L, q)$ 为语言 L 在模型 q 下的交叉熵, 这里利用了随机过程的遍历性假设去近似。

2.4 统计学习模型分类

2.4.1 生成式模型 (Generative Model)

- 建模对象：联合概率分布 $P(X, Y)$ 。
- 原理：先对 $P(X, Y)$ 建模，再利用贝叶斯公式求 $P(Y|X)$ 。
- 典型模型：n-gram, HMM, Naive Bayes。
- 特点：收敛速度快，由于学习了数据的生成机制，可以处理隐变量；但很难融合复杂的重叠特征。

2.4.2 判别式模型 (Discriminative Model)

- 建模对象：条件概率分布 $P(Y|X)$ 或直接学习决策函数 $f(X)$ 。
- 原理：直接寻找不同类别之间的最优分类面。
- 典型模型：SVM, MaxEnt, CRF, Neural Networks。
- 特点：准确率通常更高，可以灵活利用各种上下文特征 (Arbitrary overlapping features)；但训练开销通常较大。

2.5 最大熵模型 (MaxEnt)

2.5.1 核心原理

“承认无知，保留最大不确定性”：在满足所有已知约束条件（特征期望一致）的模型集合中，选择熵最大的那个模型。这里实际上指的是模型的条件熵 $H(Y|X)$ 最大。

2.6 在 NLP 的应用：词义消解

最大熵模型常用于词义消解 (Word Sense Disambiguation, WSD)，通过结合多种上下文特征，选择最合适的词义标签。为什么使用 MaxEnt？那是因为它能够灵活地整合多种特征信息，同时避免对未观察到的事件做出过于自信的预测。

2.6.1 数学定义

- 特征函数 $f_i(x, y)$ ：二值函数，描述 (x, y) 是否满足某条件。
- 约束条件：模型对特征 f_i 的期望 $E_P(f_i)$ 等于训练数据的经验期望 $\tilde{E}(f_i)$ 。

$$\sum_{x,y} \tilde{P}(x)P(y|x)f_i(x,y) = \sum_{x,y} \tilde{P}(x,y)f_i(x,y) \quad (1)$$

- 最终形式 (对数线性模型)：利用拉格朗日乘子法求解（证明可见 PPT 或者李航机器学习方法第八章），得到：

$$P_w(y|x) = \frac{1}{Z(x)} \exp \left(\sum_{i=1}^k \lambda_i f_i(x, y) \right) \quad (2)$$

其中 $Z(x)$ 为归一化因子, λ_i 为特征权重参数。 $Z(x)$ 确保对所有可能的 y 值, 条件概率和为 1。具体公式为:

$$Z(x) = \sum_y \exp \left(\sum_{i=1}^k \lambda_i f_i(x, y) \right) \quad (3)$$

经过观察, 这跟 softmax 函数的形式是一样的。更一般的, 跟多类逻辑回归是一样的。区别在于最大熵的特征是特征函数, 而逻辑回归的特征是输入向量。

2.6.2 3. 应用实例: 词义消歧 (WSD)

词义消歧是 MaxEnt 最经典的应用场景之一。

(1) **任务定义** 给定一个多义词 w 及其上下文环境 C (Context), 从候选义项集合 $S = \{s_1, s_2, \dots, s_k\}$ 中选择最合适的义项 $\hat{s}$ 。

$$\hat{s} = \arg \max_{s \in S} P(s|C)$$

(2) **特征设计 (Feature Selection)** 这是 MaxEnt 的核心优势, 可以灵活融合多种特征。假设我们要判断多义词“打”在句子中的含义。

- 上下文词特征:

$$f_1(C, s) = \begin{cases} 1, & \text{若上下文中有“篮球”且 } s = \text{“play”} \\ 0, & \text{否则} \end{cases}$$

- 词性特征:

$$f_2(C, s) = \begin{cases} 1, & \text{若“打”后面跟名词 (NN) 且 } s = \text{“hit/beat”} \\ 0, & \text{否则} \end{cases}$$

- 搭配特征:

$$f_3(C, s) = \begin{cases} 1, & \text{若宾语是“毛衣”且 } s = \text{“knit”} \\ 0, & \text{否则} \end{cases}$$

(3) **决策过程** 对于一个新的句子包含“打”, 模型根据所有激活的特征函数 f_i 及其训练好的权重 λ_i , 计算每个义项的概率:

$$P(\text{play}|C) \propto \exp(\lambda_1 \cdot 1 + \lambda_2 \cdot 0 + \dots)$$

$$P(\text{knit}|C) \propto \exp(\lambda_1 \cdot 0 + \dots + \lambda_3 \cdot 1)$$

最后取概率最大的义项。

2.6.3 4. 参数训练: 通用迭代尺度法 (GIS)

GIS (Generalized Iterative Scaling) 算法用于寻找最优参数 λ^* 。

前提条件 GIS 算法要求所有特征函数之和为常数 C :

$$\sum_i f_i(x, y) = C \quad (\text{对任意 } x, y)$$

注: 如果实际特征不满足此条件, 需定义一个松弛特征 (*Slack Feature*) f_{k+1} 补足, 使得 $\sum_{i=1}^{k+1} f_i(x, y) = C$ 。

算法流程

1. **初始化**: 设置 $\lambda_i^{(0)} = 0$ (或任意常数), $t = 0$ 。

2. **计算经验期望** (Fixed, 只需算一次):

$$\tilde{E}(f_i) = \frac{1}{N} \sum_{j=1}^N f_i(x_j, y_j)$$

3. **迭代循环** (直到收敛):

(a) 基于当前参数 $\lambda^{(t)}$, 计算**模型期望**:

$$E^{(t)}(f_i) = \sum_x \tilde{P}(x) \sum_y P_{\lambda^{(t)}}(y|x) f_i(x, y)$$

(b) **更新参数** λ_i :

$$\lambda_i^{(t+1)} = \lambda_i^{(t)} + \frac{1}{C} \ln \frac{\tilde{E}(f_i)}{E^{(t)}(f_i)} \quad (4)$$

物理意义: 如果观察到的特征次数 $\tilde{E}$ 比模型预测的 $E^{(t)}$ 多, 就增加该特征的权重 λ , 反之减少。

(c) 更新 $t \leftarrow t + 1$ 。

4. **输出**: 最优参数 λ^* 。

算法评价 GIS 算法简单易懂, 保证收敛, 但收敛速度极慢。

- **改进算法 IIS (改进迭代尺度法)**: 解决了 GIS 中 C 必须为常数的限制, 加快了步长。
- **拟牛顿法 (L-BFGS)**: 目前工程上最常用的方法, 收敛速度远远快于 GIS/IIS。

2.7 条件随机场 (CRF)

2.7.1 为什么需要 CRF?

1. **对比 HMM**: HMM 假设观察值独立 (Output Independence), 限制了特征的使用。CRF 没有此假设, 可以利用长距离上下文特征。
2. **对比 MEMM (最大熵马尔可夫)**: MEMM 存在标注偏置问题 (Label Bias Problem)。因为 MEMM 进行的是局部归一化 (每个状态单独归一), 倾向于选择出度较少的状态路径。CRF 进行全局归一化, 彻底解决了此问题。

2.7.2 线性链 CRF 模型

最常用于序列标注（如分词、词性标注、NER）。

$$P(Y|X) = \frac{1}{Z(X)} \exp \left(\sum_{i=1}^n \sum_k \lambda_k f_k(y_{i-1}, y_i, X, i) \right) \quad (5)$$

- $t_k(y_{i-1}, y_i, X, i)$: 转移特征 (依赖于前后标签关联)。
- $s_l(y_i, X, i)$: 状态特征 (依赖于当前标签与观测值的关联)。

2.7.3 CRF 的解码: Viterbi 算法

问题: 给定模型参数 λ 和观测序列 X , 找到概率最大的标记序列 $Y^* = \arg \max_Y P(Y|X)$ 。
这是一个动态规划问题。暴力搜索复杂度为 $O(N^T)$, Viterbi 复杂度为 $O(T \cdot N^2)$ 。

Algorithm 1 Viterbi 算法 (CRF 解码)

Input: 观测序列 X , 特征函数集 $\{f_k\}$, 权重 $\{\lambda_k\}$, 状态集 S (大小 m)

Output: 最优路径 Y^*

- 1: 定义 $\delta_t(j)$: 时刻 t 到达状态 j 的最大非归一化概率 (分数)。
 - 2: 定义 $\psi_t(j)$: 时刻 t 到达状态 j 的最优路径中, 前一个时刻的状态 (回溯指针)。
 - 3: **1. 初始化 (t=1)**
 - 4: **for** $j = 1 \rightarrow m$ **do**
 - 5: $\delta_1(j) = \sum_k \lambda_k f_k(y_0 = \text{start}, y_1 = j, X, 1)$
 - 6: $\psi_1(j) = 0$
 - 7: **end for**
 - 8: **2. 递推 (t=2 to T)**
 - 9: **for** $t = 2 \rightarrow T$ **do**
 - 10: **for** $j = 1 \rightarrow m$ (当前状态) **do**
 - 11: **核心公式:** 寻找前一时刻哪个状态 i 转移到 j 得分最高
 - 12: $\delta_t(j) = \max_{1 \leq i \leq m} [\delta_{t-1}(i) + \sum_k \lambda_k f_k(y_{t-1} = i, y_t = j, X, t)]$
 - 13: $\psi_t(j) = \arg \max_{1 \leq i \leq m} [\delta_{t-1}(i) + \dots]$
 - 14: **end for**
 - 15: **end for**
 - 16: **3. 终止与回溯**
 - 17: 最大分数 $P_{max} = \max_j \delta_T(j)$
 - 18: 最优路径终点 $y_T^* = \arg \max_j \delta_T(j)$
 - 19: **for** $t = T - 1 \rightarrow 1$ **do**
 - 20: $y_t^* = \psi_{t+1}(y_{t+1}^*)$
 - 21: **end for**
 - 22: **return** 序列 $Y^* = (y_1^*, \dots, y_T^*)$
-

复习重点: Viterbi 总结

Viterbi 本质就是在一个有向图 (T 列 x N 行) 中寻找一条边权重之和最大的路径。

- δ (Delta): 记录走到当前的 “最高分”。
- ψ (Psi): 记录 “从哪儿来的”，用于最后倒着找回去。

2.8 基于 CRF 的字标注分词方法

2.8.1 1. 基本思想

将分词问题转化为序列标注 (Sequence Labeling) 问题。不再直接判断词的边界，而是对句子中的每一个汉字进行分类，判断其在词语中的位置。

- 输入: 汉字序列 $X = (x_1, x_2, \dots, x_n)$
- 输出: 标签序列 $Y = (y_1, y_2, \dots, y_n)$

2.8.2 2. 标注集 (Tagset)

最常用的 4 位标注集 (BMES):

- **B (Begin)**: 词首字。
- **M (Middle)**: 词中字。
- **E (End)**: 词尾字。
- **S (Single)**: 单字词。

2.8.3 3. 标注实例

原始句子	我	爱	北	京	天	安	门
分词结果	我	爱	北京		天安门		
标注序列	S	S	B	E	B	M	E

表 1: BMES 标注示例

- “我” 是单字词 $\rightarrow$ S
- “北京” 是双字词 $\rightarrow$ “北” 是词首 (B), “京” 是词尾 (E)
- “天安门” 是三字词 $\rightarrow$ “天” (B), “安” (M), “门” (E)

2.8.4 4. 特征模板 (Feature Templates)

CRF 的强大之处在于可以利用任意的上下文特征。在字标注分词中，通常使用局部窗口内的字符及其组合作为特征。设当前字为 C_0 ，前一个字为 C_{-1} ，后一个字为 C_1 。常见的特征模板如下：

1. Unigram 特征 (单字特征):

- C_{-1} (前一个字是什么?)
- C_0 (当前字是什么?)
- C_1 (后一个字是什么?)

例子：对于“天安门”中的“安” (C_0)，特征 C_{-1} = “天”强烈暗示“安”可能是 M 或 E 。

2. Bigram 特征 (双字特征):

- $C_{-1}C_0$ (前字 + 当前字)
- C_0C_1 (当前字 + 后字)
- $C_{-1}C_1$ (跳跃组合，较少用)

例子： C_0C_1 = “安门”，这个组合经常出现在词尾，提示“门”标记为 E 。

2.8.5 5. 优势总结

相比于传统的机械分词 (FMM/BMM)，基于 CRF 的字标注方法优势在于：

- **解决未登录词 (OOV)**：模型学习的是构词规律（如“张”字常做姓氏 B ，“伟”字常做名 E ），而非死记硬背词典。因此能识别出训练集中没见过的人名、地名。
- **利用全局信息**：CRF 的全局归一化机制避免了 HMM 的局部最优陷阱。

3 N 元文法模型 (6 学时宗成庆)

复习重点: 本章核心考点

- **核心思想:** 利用马尔可夫假设 (有限历史) 来解决语言模型参数空间爆炸的问题。
- **计算能力:** 必须掌握最大似然估计 (MLE) 的计算方法, 以及如何计算句子的概率。
- **难点突破:** 数据平滑。理解“零概率”带来的危害, 以及加 1 法、Good-Turing 估计、插值法如何重新分配概率质量 (“劫富济贫”)。
- **应用场景:** 拼音输入法 (音字转换) 是 N-gram 最直观的应用。

3.1 模型定义 (Model Definition)

3.1.1 1. 语言模型与链式法则

语言模型 (Language Model, LM) 的任务是计算一个句子 (词序列) $S = w_1, w_2, \dots, w_m$ 的概率 $P(S)$ 。根据概率论的**链式法则 (Chain Rule)**, 联合概率可以分解为条件概率的乘积:

$$P(S) = P(w_1) \times P(w_2|w_1) \times P(w_3|w_1w_2) \times \dots \times P(w_m|w_1 \dots w_{m-1}) \quad (6)$$

问题: 随着句子变长, 历史上下文 $w_1 \dots w_{i-1}$ 的组合空间呈指数级爆炸, 导致参数无法估计 (数据稀疏)。

3.1.2 2. 马尔可夫假设 (Markov Assumption)

为了解决参数爆炸问题, 引入马尔可夫假设: **当前词出现的概率只与前面 $n-1$ 个词相关。**

$$P(w_i|w_1 \dots w_{i-1}) \approx P(w_i|w_{i-n+1} \dots w_{i-1}) \quad (7)$$

3.1.3 3. 常见 N-gram 模型

- **Unigram (一元文法, $n=1$):** 词与词相互独立。

$$P(S) \approx \prod_{i=1}^m P(w_i)$$

- **Bigram (二元文法, $n=2$):** 只看前 1 个词 (一阶马尔可夫链)。

$$P(S) \approx \prod_{i=1}^m P(w_i|w_{i-1})$$

- **Trigram (三元文法, $n=3$):** 只看前 2 个词 (二阶马尔可夫链)。

$$P(S) \approx \prod_{i=1}^m P(w_i|w_{i-2}w_{i-1})$$

复习重点: 关于 BOS 和 EOS

为了使概率分布在句子层面上归一化 (即所有可能长度的句子概率之和为 1), 通常在句首添加 $\langle BOS \rangle$ (Begin Of Sentence), 在句尾添加 $\langle EOS \rangle$ (End Of Sentence)。

3.2 参数估计 (Parameter Estimation)

3.2.1 最大似然估计 (MLE)

利用训练语料中的频率计数来估计概率。对于 N-gram 模型，其参数估计公式为：

$$P_{MLE}(w_i|w_{i-n+1}^{i-1}) = \frac{Count(w_{i-n+1}^{i-1}w_i)}{Count(w_{i-n+1}^{i-1})} \quad (8)$$

即：(历史 + 当前词) 的共现次数除以 (历史) 的出现次数。

3.2.2 实例计算 (The "John read Moby Dick" Example)

假设训练语料库包含以下 3 个句子：

1. $\langle BOS \rangle$ John read Moby Dick $\langle EOS \rangle$
2. $\langle BOS \rangle$ Mary read a different book $\langle EOS \rangle$
3. $\langle BOS \rangle$ She read a book by Cher $\langle EOS \rangle$

Bigram 概率计算示例：

- 计算 $P(read|John)$ ：

- $Count(John) = 1$
- $Count(John, read) = 1$
- $\Rightarrow P(read|John) = 1/1 = 1$

- 计算 $P(a|read)$ ：

- $Count(read) = 3$ (出现在句 1, 2, 3 中)
- $Count(read, a) = 2$ (出现在句 2, 3 中，分别是"read a")
- $\Rightarrow P(a|read) = 2/3$

- 计算 $P(book|a)$ ：

- $Count(a) = 2$
- $Count(a, book) = 1$ (句 3: "a book")，注意句 2 是"a different"
- $\Rightarrow P(book|a) = 1/2$

3.3 数据平滑 (Data Smoothing)

3.3.1 为什么要平滑？

数据稀疏 (Data Sparsity) 是 N-gram 的致命伤。

- **零概率问题：**如果测试集中出现了训练集中未见过的 N-gram (如"Cher read")，MLE 会赋予其 0 概率。

- **后果：**由于连乘效应，只要有一个词的概率为 0，整句话的概率 $P(S)$ 就变成 0。这显然是不合理的。
- **核心思想：“劫富济贫”。**将高频事件的概率量（富）削减一部分，分配给未见过的事件（贫）。

3.3.2 1. 加 1 平滑 (Add-one / Laplace Smoothing)

最简单的平滑方法。假设每个 N-gram 在训练集中多出现了一次。

$$P_{Add1}(w_i|w_{i-1}) = \frac{C(w_{i-1}w_i) + 1}{C(w_{i-1}) + |V|} \quad (9)$$

- $|V|$ ：词表大小 (Vocabulary Size)。
- **缺点：**通常 $|V|$ 很大（几万到几十万），导致分母剧增，原始的非零概率被严重稀释（削减太多），效果往往不佳。

3.3.3 2. Good-Turing 估计 (重点理解)

这是许多平滑算法（如 Katz Back-off）的基础。它利用频率的频率来估计概率。

- **定义：**Let N_r be the number of n-grams that occur r times. (出现 r 次的 n-gram 有 N_r 个)。
- **调整计数 r^* ：**对于出现 r 次的事件，认为其实际发生次数应修正为：

$$r^* = (r + 1) \frac{N_{r+1}}{N_r} \quad (10)$$

- **概率估计：**

$$P_r = \frac{r^*}{N}$$

- **未见事件 ($r = 0$) 的概率：**

$$P_0 = \frac{N_1}{N}$$

即：所有未见过的 n-gram 的总概率，等于出现一次的 n-gram 在语料中的总比例。

3.3.4 3. 线性插值法 (Linear Interpolation)

简单且有效。将高阶模型（如 Trigram）与低阶模型（Bigram, Unigram）进行加权组合。

$$\hat{P}(w_n|w_{n-2}w_{n-1}) = \lambda_3 P(w_n|w_{n-2}w_{n-1}) + \lambda_2 P(w_n|w_{n-1}) + \lambda_1 P(w_n) \quad (11)$$

- **约束：** $\sum \lambda_i = 1$ 。
- 能够充分利用高阶模型的准确性和低阶模型的鲁棒性。若高阶数据稀疏 (Count=0)，低阶模型能提供非零概率支持。

3.4 模型评价

困惑度 (Perplexity, PP) 是衡量语言模型好坏的标准指标。

- 定义：测试集概率的倒数的几何平均。

$$PP(S) = P(w_1 \dots w_N)^{-\frac{1}{N}} = \sqrt[N]{\frac{1}{P(w_1 \dots w_N)}} \quad (12)$$

- 与熵的关系： $PP(S) = 2^{H(S)}$ ，其中 $H(S)$ 是交叉熵。
- 直观含义：模型在预测下一个词时，平均有多少个等可能的选择（分支系数）。
- 标准：PP 值越小，模型越好。

3.5 典型应用：音字转换 (Pinyin-to-Character)

拼音输入法本质上是一个解码问题。

- 输入：拼音串 S (如"ta shi yan jiu sheng wu de")。
- 输出：汉字串 W 。
- 公式 (基于噪声信道模型)：

$$\hat{W} = \arg \max_W P(W|S) = \arg \max_W \frac{P(S|W)P(W)}{P(S)} \approx \arg \max_W P(W)$$

因为给定汉字 W ，其拼音 S 是固定的（忽略多音字模糊性），即 $P(S|W) \approx 1$ 。因此，问题转化为寻找语言模型概率 $P(W)$ 最大的汉字序列。

- 示例分析：
 - 候选 1：踏实 (adj) 研究 (v) 生物 (n) 的 (u)
 - 候选 2：他 (pron) 是 (v) 研究 (v) 生物 (n) 的 (u)

在 Bigram 模型中，比较关键路径概率：

- $P(\text{是}|\text{他})$ 通常远大于 $P(\text{研究}|\text{踏实})$ 。
- 因此语言模型会正确选择“他是研究生物的”。

4 传统 NLP 关键技术 (6 学时宗成庆)

复习重点: 章节导读

本章横跨 300 页 PPT，涵盖了深度学习时代之前的 NLP 三大基石：词法、句法与语义。
核心考点：

- 算法手推：CKY 算法填充解析表、Shift-Reduce 的动作序列推导。
- 歧义辨析：交集型 vs 组合型歧义；结构歧义的消解。
- 模型对比：生成式 (HMM/PCFG) vs 判别式 (CRF/Dependency)。

4.1 词语切分与命名实体识别 (Lexical Analysis)

4.1.1 1. 分词难点：歧义与未登录词

1. 切分歧义 (Segmentation Ambiguity):

- 交集型歧义 (Intersection Ambiguity): 字符串 ABC 中, AB 成词, BC 也成词。
例：“结合成分子” → “结合/成/分子” vs “结合/成分/子”。定义：交集串链的长度是衡量处理难度的指标。
- 组合型歧义 (Combination Ambiguity): 字符串 AB 在某些语境下是词, 在另一些语境下是两个词。例：“这个人手脚不干净”（词：偷窃）vs “冻得手脚冰凉”（短语：手和脚）。

2. 未登录词 (OOV) 识别：包括人名（中国人名/外国人名译名）、地名、机构名、新词（如“奥利给”）。OOV 是造成分词错误的主要原因（占 >60%）。

4.1.2 2. 机械切分算法 (Rule-based)

假设：词典是完备的。

Algorithm 2 正向/逆向最大匹配算法

- FMM (Forward Maximum Matching): 从左向右扫描，取窗口 $MaxLen$ ，查词典。若不匹配，去掉最后一个字，继续查。缺点：倾向于切分出短词，对于偏正结构（重心在后）处理较差。
 - BMM (Backward Maximum Matching): 从右向左扫描。对于汉语，BMM 准确率通常高于 FMM（约 1% 的差异）。
 - 双向匹配 (Bi-directional Matching): 同时运行 FMM 和 BMM。决策规则：1. 若结果相同，输出任意一个。2. 若结果不同，选择词数量较少的结果（长词优先）。3. 若词数也相同，选择单字词较少的结果。
-

4.1.3 3. 统计分词方法 (Statistical)

将分词建模为字位标注 (Sequence Labeling) 问题。

- 标签集 (Tagset): {B (Begin), M (Middle), E (End), S (Single)}。
- 模型对比:
 - **HMM**: $P(X, Y) = \prod P(y_i | y_{i-1}) P(x_i | y_i)$ 。生成式, 假设独立, 难以利用长距离特征。
 - **CRF**: $P(Y | X) = \frac{1}{Z} \exp(\sum \lambda f(y, x))$ 。判别式, 全局归一化, 能利用丰富的上下文特征, 是传统分词的 SOTA 方法。

4.1.4 4. 子词切分 (Subword Segmentation)

针对英语等印欧语系, 解决 OOV 和词表过大问题。

- **BPE (Byte Pair Encoding)**: 迭代合并语料中频率最高的字符对 (Character Pair), 直到达到预设的词表大小。作用: 自动学习词根、词缀 (如 "un-", "-ing"), 实现 “开放词汇” 覆盖。

4.1.5 5. 词性标注与命名实体识别

- 标注集示例:
 - 北大标准 (PKU): /n (名词), /v (动词), /vn (名动词), /w (标点)。
 - 宾州树库 (Penn Treebank): NN, VB, JJ, DT 等。
- 实体类型 (NER): PER (人名), LOC (地名), ORG (机构名)。例句:

克拉伦斯/nrg • /w 威姆斯/nrf (Clarence Weems) 和/c 张/nrf 宁/nrg 相继/d 上
篮/v 得手/v

4.2 句法分析 (Syntactic Parsing)

4.2.1 1. 短语结构分析 (Phrase Structure Parsing)

- 文法形式: PCFG (概率上下文无关文法)。规则形式: $A \rightarrow \beta[p]$, 其中 $\sum_{\beta} P(A \rightarrow \beta) = 1$ 。
- 核心算法: CKY 算法 (Cocke-Younger-Kasami)
 - 前提: 文法必须转换为 CNF (乔姆斯基范式), 即 $A \rightarrow BC$ 或 $A \rightarrow w$ 。
 - 原理: 动态规划。构建 $(n+1) \times (n+1)$ 的三角矩阵。
 - 递推公式: 对于跨度 $[i, j]$ 和分裂点 k ($i < k < j$):

$$P(A_{i,j}) = \max_{k,B,C} P(B_{i,k}) \cdot P(C_{k,j}) \cdot P(A \rightarrow BC)$$

- 复杂度: $O(n^3 \cdot |G|^3)$ 。
- PCFG 的三个基本问题 (类比 HMM): 1. 计算句子概率 (Inside Algorithm); 2. 寻找最优句法树 (Viterbi Algorithm); 3. 参数估计 (Inside-Outside Algorithm)。

4.2.2 2. 依存句法分析 (Dependency Parsing)

核心：词与词之间的二元非对称关系 (Head $\rightarrow$ Modifier)。

- **约束条件：**1. 单一中心词 (Single Head); 2. 连通性 (Connected); 3. 无环 (Acyclic); 4. 投射性 (Projectivity) (如果不满足则为非投射, 常见于德语等, 需特殊处理)。
- **主流算法：**
 1. **基于图 (Graph-based): MST 算法** (Maximum Spanning Tree)。寻找有向图中的最大生成树。特点：全局最优, 能处理非投射, 但复杂度较高 ($O(n^3)$ 或 $O(n^2)$)。
 2. **基于转移 (Transition-based): Shift-Reduce 算法** (如 Arc-Eager, Arc-Standard)。
 - **数据结构：**栈 (Stack σ), 缓冲队列 (Buffer β), 依存弧集合 (A)。
 - **动作 (Actions):**
 - * **Shift:** 将 β 队首词压入 σ 。
 - * **Left-Arc:** σ 栈顶词作为子节点, β 队首词作为父节点。
 - * **Right-Arc:** σ 栈顶词作为父节点, β 队首词作为子节点。
 - * **Reduce:** 弹出 σ 栈顶。
 - **特点:** $O(n)$ 线性复杂度, 速度快, 但贪心搜索容易累积误差 (Error Propagation)。

4.2.3 3. 附录：汉英句法结构对比 (重点)

- **英语 (Hypotaxis - 形合):** “结构型”语言。句子结构严谨, 主谓宾完备。长句通过显式的连接词 (Relation words) 构造复杂的树状结构。
- **汉语 (Parataxis - 意合):** “语义型”语言。重意不重形, 常省略主语或连接词。**特有现象：**流水句 (Run-on sentences) / 句群。**难点：**句法分析器很难判断逗号是分割两个分句 (并列), 还是分割从句 (嵌套)。

4.3 语义分析 (Semantic Analysis)

4.3.1 1. 语义歧义 (Semantic Ambiguity)

- **一词多义：**Polysemy。
- **结构导致语义歧义：**“咬死了猎人的狗” (Subject/Object Ambiguity)。
- **反义同解现象：**
 - “中国队大胜美国队” vs “中国队大败美国队” $\rightarrow$ 都是中国队赢。
 - “小心火烛” vs “小心地滑” $\rightarrow$ 语义指向不同。
 - “冬天能穿多少穿多少” (多) vs “夏天能穿多少穿多少” (少)。

4.3.2 2. 语义网络 (Semantic Network)

- **WordNet**: 基于心理语言学的词汇库。节点是 **Synset** (同义词集)。
- 语义关系:
 - **Hypernym** (上位词) / **Hyponym** (下位词): is-a 关系 (Bird $\leftrightarrow$ Robin)。
 - **Holonym** (整体) / **Meronym** (部分): part-of 关系 (Car $\leftrightarrow$ Wheel)。
 - **Antonym** (反义词)。

4.3.3 3. 语义角色标注 (SRL)

任务: 浅层语义分析, 识别谓词 (Predicate) 的论元 (Arguments)。

- 两大体系:
 - **FrameNet**: 基于框架语义学。标签如 *Buyer*, *Goods*, *Money*。
 - **PropBank** (命题库): 基于 Penn Treebank。标签为 *Arg0* (施事), *Arg1* (受事), *ArgM - TMP* (时间), *ArgM - LOC* (地点)。
- 示例分析:

警察 (*Arg0/Agent*) 已 到 (*Pred*) 现场 (*Arg1/Loc*), 正在 详细 (*ArgM - MNR*) 调查 (*Pred*) 事故原因 (*Arg1/Patient*)。

5 神经网络与语言模型 (6 学时张家俊)

复习重点: 本章复习总纲

本章内容横跨 113 页 PPT，是 NLP 从统计时代迈向深度学习时代的里程碑。**核心逻辑链**：

1. **起点**：N-gram 的稀疏性与维数灾难 → **FFNN** (引入词向量，连续空间建模)。
2. **进阶**：FFNN 的固定窗口限制 → **RNN** (引入隐状态，处理变长序列)。
3. **修正**：RNN 的梯度消失问题 → **LSTM** (引入门控机制，记忆长距离信息)。
4. **终局**：RNN 的串行计算瓶颈 → **Self-Attention** (并行计算，全局视野)。

5.1 1. 神经网络基础 (Neural Networks Overview)

5.1.1 1.1 神经元模型 (M-P Model)

1943 年，McCulloch 和 Pitts 提出形式化数学描述：

$$y = f(\mathbf{W}^T \mathbf{X} + b) = f\left(\sum_{i=1}^n w_i x_i + b\right) \quad (13)$$

其中 $f(\cdot)$ 为激活函数，提供非线性能力。

5.1.2 1.2 核心激活函数

- **Sigmoid**: $\sigma(z) = \frac{1}{1+e^{-z}}$ 。输出 $(0, 1)$ ，易导致梯度消失。
- **Tanh**: $\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$ 。输出 $(-1, 1)$ ，零中心化，但仍有梯度消失问题。
- **ReLU**: $\max(0, z)$ 。解决梯度消失，计算快，是现代深度学习的首选。

5.2 2. 前馈神经网络语言模型 (FFNN-LM)

5.2.1 2.1 动机：解决 N-gram 的两大痛点

1. **数据稀疏 (Data Sparsity)**：N-gram 无法处理未见过的词组（概率为 0）。
2. **缺乏泛化 (Lack of Generalization)**：离散的 One-hot 表示使得 $\text{Sim}(\text{cat}, \text{dog}) = 0$ 。FFNN 通过词向量 (**Distributed Representation**) 将词映射到连续空间，相似词距离近，从而实现“由 cat 泛化到 dog”。

5.2.2 2.2 模型架构 (Bengio et al., 2003)

输入前 $n - 1$ 个词，预测第 n 个词。

- **输入层**：查表得到词向量 $C(w_t)$ 。
- **投影层**：拼接上下文向量 $\mathbf{x} = [C(w_{t-n+1}); \dots; C(w_{t-1})]$ 。

- 隐藏层: $\mathbf{h} = \tanh(\mathbf{H}\mathbf{x} + \mathbf{d})$ 。
- 输出层: $\mathbf{y} = \mathbf{U}\mathbf{h} + \mathbf{W}\mathbf{x} + \mathbf{b}$ (含直连边 $\mathbf{W}\mathbf{x}$)。
- 概率计算: $P(w_t|\text{context}) = \text{Softmax}(\mathbf{y})_t$ 。

5.2.3 2.3 反向传播推导 (Back-Propagation Details)

(PPT 25-30 页详细推导了梯度计算, 这是考点) 定义损失函数 $E = \frac{1}{2} \sum (t_i - y_i)^2$ (或交叉熵), 利用链式法则:

- 输出层梯度: $\delta_i = \frac{\partial E}{\partial y_i} \cdot f'(s_i) = -(t_i - y_i)y_i(1 - y_i)$ (假设 Sigmoid)。
- 隐藏层权重更新: $\Delta w_{i \leftarrow j} = \eta \delta_i h_j$ 。
- 误差反传至隐藏层: $\frac{\partial E}{\partial h_j} = \sum_i \delta_i w_{i \leftarrow j}$ 。

5.3 3. 循环神经网络语言模型 (RNN-LM)

5.3.1 3.1 核心公式

RNN 引入隐状态 h_t (Hidden State) 作为记忆单元, 处理变长序列。

$$h_t = \tanh(\mathbf{U}x_t + \mathbf{W}h_{t-1}) \quad (14)$$

$$y_t = \text{Softmax}(\mathbf{V}h_t) \quad (15)$$

特点: 权重矩阵 $\mathbf{U}, \mathbf{W}, \mathbf{V}$ 在所有时间步共享。

5.3.2 3.2 BPTT 与梯度问题

训练算法为随时间反向传播 (BPTT)。

- 梯度消失 (Gradient Vanishing): $\frac{\partial h_t}{\partial h_k} = \prod \mathbf{W}$ 。若 $\mathbf{W}$ 特征值 < 1 , 连乘导致梯度趋零, 模型无法捕捉长距离依赖。
- 梯度爆炸 (Gradient Exploding): 若特征值 > 1 , 梯度指数级增长。解法: 梯度裁剪 (Gradient Clipping)。

5.4 4. 长短时记忆网络 (LSTM)

5.4.1 4.1 核心机制: 三门一胞

LSTM 通过引入门控机制和独立的细胞状态 C_t (Cell State) 解决梯度消失。(PPT 75-82 页详细公式):

1. 遗忘门 (Forget Gate): 决定丢弃多少旧记忆。

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f)$$

2. 输入门 (Input Gate): 决定写入多少新信息。

$$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i)$$

3. 候选记忆 (Candidate Cell): 生成新的待存信息。

$$\tilde{C}_t = \tanh(W_C[h_{t-1}, x_t] + b_C)$$

4. 细胞状态更新 (核心): 加法更新形成“梯度高速公路”。

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$$

5. 输出门 (Output Gate): 控制隐状态输出。

$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o)$$

6. 隐状态计算:

$$h_t = o_t \odot \tanh(C_t)$$

5.5 5. 自注意力机制 (Self-Attention)

5.5.1 5.1 动机

RNN 只能串行计算 (t 依赖 $t-1$), 且长距离依赖仍有损耗。Attention 实现并行计算且路径长度为 $O(1)$ 。

5.5.2 5.2 计算步骤 (Scaled Dot-Product Attention)

(PPT 90-102 页详细推导) 给定输入序列 $\mathbf{X}$:

1. 线性映射: 生成 Query ($\mathbf{Q}$), Key ($\mathbf{K}$), Value ($\mathbf{V}$)。

$$\mathbf{Q} = \mathbf{XW}^Q, \quad \mathbf{K} = \mathbf{XW}^K, \quad \mathbf{V} = \mathbf{XW}^V$$

2. 计算相关性 (Attention Scores):

$$e_{jk} = \mathbf{q}_j \cdot \mathbf{k}_k^T$$

3. 缩放与归一化: 除以 $\sqrt{d_k}$ 防止梯度消失, 再经 Softmax。

$$\alpha_{jk} = \frac{\exp(e_{jk}/\sqrt{d_k})}{\sum_{k'} \exp(e_{jk'}/\sqrt{d_k})}$$

4. 加权求和:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \sum_k \alpha_{jk} \mathbf{v}_k = \text{softmax} \left(\frac{\mathbf{QK}^T}{\sqrt{d_k}} \right) \mathbf{V}$$

5.5.3 5.3 多头注意力 (Multi-Head Attention)

将 $\mathbf{Q}, \mathbf{K}, \mathbf{V}$ 投影到 h 个不同的子空间并行计算，最后拼接 (Concat) 输出。让模型能同时关注不同的特征（如局部语法 vs 全局语义）。

$$\text{MultiHead}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) \mathbf{W}^O$$

5.5.4 5.4 GPT (Generative Pre-trained Transformer)

基于 Transformer **Decoder** 的单向语言模型。通过在大规模语料上进行无监督预训练 (Pre-training)，学习通用的语言表示，再在下游任务微调 (Fine-tuning)。

6 文本表示 (Text Representation) (3 学时张家俊)

复习重点: 本章复习总纲

文本表示是 NLP 的基石。复习时请重点把握从“离散”到“分布式”的演变逻辑：

1. 离散表示 (Discrete): One-hot, BoW, TF-IDF。核心缺陷是维度爆炸和语义鸿沟。
2. 分布式表示 (Distributed): 核心是词向量 (Word Embedding)。
 - 基于语言模型: NNLM (Bengio) 的副产品。
 - 直接学习法: C&W, Word2Vec (CBOW/Skip-gram), GloVe。
3. 句子/文档表示: 从简单的词向量平均到 CNN/RNN/Attention 建模。

6.1 1. 向量空间模型 (Vector Space Model, VSM)

6.1.1 1.1 基本概念

由 Salton 等人于 60 年代提出，核心是将文本视为特征项 (Term) 的集合。

- 表示: 文本 d 表示为权重向量 $\mathbf{d} = (w_1, w_2, \dots, w_N)$ 。
- 特征项: 通常是词 (Word) 或短语。
- 权重计算:
 - **TF (Term Frequency):** 词频。 $w_i = TF_i$ 。
 - **IDF (Inverse Document Frequency):** 逆文档频率。 $IDF_i = \log \frac{N}{df_i}$ (N 为总文档数, df_i 为包含特征项 t_i 的文档数)。
 - **TF-IDF:** $w_i = TF_i \times IDF_i$ 。降低常见词 (如 "the") 的权重, 提升独特词的权重。

6.1.2 1.2 长度规范化

长文档可能包含更高的词频，导致权重偏差。

$$\mathbf{d}_{norm} = \frac{\mathbf{d}}{\|\mathbf{d}\|_2} = \frac{(w_1, \dots, w_N)}{\sqrt{\sum w_i^2}} \quad (16)$$

6.1.3 1.3 离散符号表示的缺陷

- **One-hot 表示**：维度等于词表大小 $|V|$ ，极其稀疏。
- **语义鸿沟**：任意两个不同词的 One-hot 向量正交 ($\mathbf{w}_i \cdot \mathbf{w}_j = 0$)，无法计算相似度。

6.2 2. 词语的表示学习 (Word Embedding)

6.2.1 2.1 分布式假设 (Distributional Hypothesis)

"You shall know a word by the company it keeps." (J.R. Firth, 1957)

词的语义由其上下文决定。

6.2.2 2.2 基于语言模型的方法

NNLM (Bengio et al., 2003):

- 目标是最大化语言模型似然度 $L = \sum_t \log P(w_t | w_{t-n+1}^{t-1})$ 。
- 词向量矩阵 $\mathbf{L} \in \mathbb{R}^{|V| \times D}$ 是模型的参数，随训练过程自动学习优化。
- **地位**：词向量是语言模型任务的“副产品”。

6.2.3 2.3 C&W 模型 (Collobert & Weston, 2008)

核心思想：将目标从预测下一个词改为区分正例和负例 (Ranking Loss)。

- **输入**：
 - 正例 s ：合法句子 (Context + Target Word)。
 - 负例 s' ：将 Target Word 随机替换为词表中的其他词。
- **目标函数**：Hinge Loss (Max-margin)。

$$Loss = \max(0, 1 + Score(s') - Score(s)) \quad (17)$$

要求正例的打分比负例高至少 1。

6.2.4 2.4 Word2Vec (Mikolov et al., 2013) [核心考点]

Word2Vec 去掉了非线性的隐藏层，直接训练词向量，极大提高了训练速度。

(1) CBOW (Continuous Bag-of-Words)

- 任务：根据上下文预测中心词。
- 输入：上下文 $2C$ 个词向量的平均值 (或求和)。

$$\mathbf{h} = \frac{1}{2C} \sum_{j \neq 0} \mathbf{v}_{w_{t+j}}$$

- 目标：最大化 $P(w_t | context)$ 。

(2) Skip-gram

- 任务：根据中心词预测上下文词。
- 输入：中心词向量 $\mathbf{v}_{w_t}$ 。
- 目标：最大化 $\sum_{j \neq 0} \log P(w_{t+j} | w_t)$ 。

(3) 训练加速技巧

- 层次化 Softmax (Hierarchical Softmax)：利用 Huffman 树将 $O(|V|)$ 降为 $O(\log |V|)$ 。
- 负采样 (Negative Sampling)：只更新正样本和少量随机采样的负样本。

$$\log \sigma(\mathbf{v}_w^T \mathbf{v}_c) + \sum_{k=1}^K \log \sigma(-\mathbf{v}_{neg_k}^T \mathbf{v}_c)$$

6.2.5 2.5 GloVe (Global Vectors)

动机：结合全局矩阵分解 (LSA) 和局部上下文窗口 (Word2Vec) 的优点。

- 输入：词共现矩阵 $\mathbf{X}$ ，其中 X_{ij} 表示词 i 和词 j 共现的次数。
- 目标函数：加权最小二乘误差。

$$J = \sum_{i,j} f(X_{ij})(\mathbf{w}_i^T \tilde{\mathbf{w}}_j + b_i + \tilde{b}_j - \log X_{ij})^2 \quad (18)$$

- 直觉：词向量的点积 $\mathbf{w}_i^T \mathbf{w}_j$ 应逼近共现概率的对数。

6.3 3. 短语与句子表示学习

6.3.1 3.1 基础组合方法

- 向量加和/平均： $\mathbf{v}_{sent} = \frac{1}{N} \sum \mathbf{v}_{word}$ 。最简单，丢失语序信息。
- 加权平均：利用 TF-IDF 作为权重。

6.3.2 3.2 基于神经网络的方法

- **RNN/LSTM**: 取最后一个时间步的隐状态 $\mathbf{h}_T$ 作为句子表示。
- **CNN (卷积神经网络)**:
 - 利用卷积核 (Filter) 捕捉不同长度的 N-gram 特征 (如 2-gram, 3-gram)。
 - 经过 **Max Pooling** 提取最显著特征, 拼接成句子向量。
- **Recursive NN (递归神经网络)**: 在句法树结构上递归组合向量 (Socher et al.)。

6.4 4. 文档表示学习

6.4.1 4.1 层次化注意力模型 (HAN)

文档结构具有层次性: 词 $\rightarrow$ 句子 $\rightarrow$ 文档。

- **Word Encoder**: 双向 GRU 编码词序列。
- **Word Attention**: 区分词的重要性, 加权生成句子向量。
- **Sentence Encoder**: 双向 GRU 编码句子序列。
- **Sentence Attention**: 区分句子的重要性, 加权生成文档向量。

6.4.2 4.2 Doc2Vec (Paragraph Vector)

扩展了 Word2Vec。

- **PV-DM (Distributed Memory)**: 类似于 CBOW。预测词时, 输入包含上下文词向量和段落 ID 向量。
- **PV-DBOW**: 类似于 Skip-gram。根据段落 ID 预测该段落中的随机词。

7 大语言模型及其训练 (6 学时张家俊)

复习重点: 本章知识图谱

本章内容极多, 涵盖了 LLM 全生命周期的核心技术, 请按以下逻辑主线复习:

1. 模型架构 (Model): 从 ELMo $\rightarrow$ GPT $\rightarrow$ BERT $\rightarrow$ LLM (Decoder-only) 的演进。
2. 数据工程 (Data): CommonCrawl $\rightarrow$ 清洗/去重/配比 (DoReMi)。
3. 训练系统 (System): 3D 并行 (数据/张量/流水线) + 显存优化 (ZeRO, FlashAttention)。
4. 微调对齐 (Alignment): SFT $\rightarrow$ RLHF (PPO) $\rightarrow$ DPO (直接偏好优化)。
5. 提示学习 (Prompt): In-context Learning $\rightarrow$ Chain-of-Thought (CoT)。

7.1 1. 预训练语言模型演进

7.1.1 1.1 ELMo (Embeddings from Language Models)

核心贡献: 提出了上下文相关 (Contextualized) 的词向量, 解决了 Word2Vec 多义词无法区分的问题。

- 架构: 双向 LSTM (Bi-LSTM)。
- 目标函数: 最大化双向似然概率。

$$\sum_{k=1}^N (\log P(t_k | t_1, \dots, t_{k-1}) + \log P(t_k | t_{k+1}, \dots, t_N))$$

- 表示: 将各层 LSTM 的输出进行线性加权组合。

7.1.2 1.2 GPT (Generative Pre-Training)

核心贡献: 确立了 Decoder-only 架构和 Pre-train + Fine-tune 范式。

- 架构: 基于 Transformer Decoder 的单向语言模型 (Masked Self-Attention)。
- 目标: 预测下一个词 (Next Token Prediction)。

$$L_1(U) = \sum_i \log P(u_i | u_{i-k}, \dots, u_{i-1}; \Theta)$$

- 演进: GPT-1 (微调) $\rightarrow$ GPT-2 (Zero-shot, "Task description") $\rightarrow$ GPT-3 (Few-shot, In-context Learning)。

7.1.3 1.3 BERT (Bidirectional Encoder Representations from Transformers)

核心贡献：引入双向上下文和 **Masked LM (MLM)** 任务。

- 架构：Transformer Encoder。
- 预训练任务：
 1. **MLM**：随机 Mask 15% 的词，利用上下文预测被 Mask 的词。
 2. **NSP (Next Sentence Prediction)**：判断句子 B 是否是句子 A 的下文。
- 缺陷：预训练时的 [MASK] 标记在微调时不存在，导致预训练-微调不一致 (**Pre-train/Fine-tune Discrepancy**)；无法像 GPT 那样生成长文本。

7.2 2. 训练数据工程

7.2.1 2.1 数据来源 (Sources)

高质量数据是 LLM 能力的基石 (“Garbage In, Garbage Out”)。

- 网页数据 (**Common Crawl**)：体量最大 (PB 级)，但噪声极多。需经过严格清洗。
- 高质量文本：Wikipedia, BooksCorpus (书籍), arXiv (论文)。
- 代码数据：GitHub (提升模型的逻辑推理能力)。
- 对话数据：Reddit, Social Media (提升对话能力)。

7.2.2 2.2 数据处理流水线 (Pipeline)

1. 质量过滤 (Quality Filtering):

- 启发式规则：过滤过短/过长句子，过滤特殊符号过多的文本，过滤困惑度 (Perplexity) 过高的文本。
- 基于分类器：训练一个二分类器 (如 fastText)，区分 “高质量” 与 “低质量” 网页。

2. 去重 (De-duplication):

- 精确去重：MD5/SHA-1 哈希比对。
- 模糊去重 (**Fuzzy Dedup**): 使用 **MinHash + LSH (Locality Sensitive Hashing)** 算法，检测相似度高的文档 (如 Jaccard 相似度 > 0.8)。作用：防止模型背诵训练集，提升泛化能力。

3. 隐私去除 (**Privacy Redaction**): 使用正则或 NER 模型去除 PII (Personal Identifiable Information)。

7.2.3 2.3 数据配比 (Data Mixture)

不同来源的数据占比对模型性能影响巨大。

- **DoReMi 算法 (Xie et al., 2023)**: 利用一个小规模代理模型 (Proxy Model) 在目标域上通过群组分布鲁棒优化 (**Group DRO**) 自动搜索各数据域的最佳权重, 然后再用该权重训练大模型。结论: 异质数据来源比数据规模更重要。

7.3 3. 高效并行训练方法

7.3.1 3.1 显存占用分析

训练一个参数量为 Φ 的模型, 显存占用远大于 2Φ (FP16)。显存占用公式:

$$\text{Total Memory} \approx \text{模型参数} + \text{梯度} + \text{优化器状态} + \text{激活值}$$

对于 Adam 优化器混合精度训练:

- **模型参数 (FP16)**: 2Φ
- **梯度 (FP16)**: 2Φ
- **优化器状态 (FP32)**: 参数备份 (4Φ) + Momentum (4Φ) + Variance (4Φ) = 12Φ
- **总计静态显存**: $2 + 2 + 12 = 16\Phi$ 字节。例: $175B$ 的 *GPT-3* 仅静态显存就需要 $175 \times 10^9 \times 16B \approx 2.8TB$, 单卡无法放下。

7.3.2 3.2 3D 并行策略 (3D Parallelism)

1. **数据并行 (Data Parallelism, DP)**: 多卡复制模型, 切分数据。每卡计算梯度, 通过 **All-Reduce** 同步梯度。
2. **张量并行 (Tensor Parallelism, TP)** (Megatron-LM): 层内切分。将矩阵乘法 $Y = XA$ 切分到多卡。
 - **列切分 (Column Parallel)**: 将 A 按列切分, 各卡计算部分 Y , 最后 Concat。
 - **行切分 (Row Parallel)**: 将 A 按行切分, 输入 X 按列切分, 各卡计算后 All-Reduce 求和。
3. **流水线并行 (Pipeline Parallelism, PP)** (GPipe/PipeDream): 层间切分。将模型的不同层放置在不同 GPU 上。
 - **气泡问题 (Bubble)**: GPU 等待前序数据导致空闲。
 - **1F1B 策略**: 交替执行 1 个 Forward 和 1 个 Backward, 减少空闲时间。

7.3.3 3.3 显存优化技术

- **ZeRO (Zero Redundancy Optimizer)** (DeepSpeed): 核心思想是切分数据并行的冗余状态。
 - **ZeRO-1**: 切分优化器状态 (P_{os})。显存降为 4Φ 。
 - **ZeRO-2**: 切分梯度 ($P_{os} + P_g$)。显存降为 2Φ 。
 - **ZeRO-3**: 切分模型参数 ($P_{os} + P_g + P_p$)。显存线性减少, 通信量增加 50%。
 - **ZeRO-Offload**: 将优化器状态卸载到 CPU 内存。
- **FlashAttention** (Dao et al., 2022): **IO 感知优化**。通过分块计算 (Tiling) 和重计算 (Re-computation), 将 Attention 的 $O(N^2)$ 显存复杂度降为 $O(N)$, 并大幅减少 HBM (高带宽显存) 到 SRAM 的读写次数, 训练速度提升 2-4 倍。

7.4 4. 微调与对齐

7.4.1 4.1 指令微调 (Instruction Tuning)

目的: 让模型学会遵循人类指令 (Follow Instructions), 而不仅仅是续写文本。

- **方法**: 构建 (Instruction, Input, Output) 三元组数据进行有监督微调 (SFT)。
- **数据集**: FLAN, P3, Self-Instruct (用模型生成指令数据)。

7.4.2 4.2 基于人类反馈的强化学习 (RLHF)

流程 (3H 原则: Helpful, Honest, Harmless):

1. **SFT (Supervised Fine-Tuning)**: 在高质量指令数据上微调基座模型 π_{SFT} 。
2. **RM (Reward Modeling)**: 训练奖励模型 $r_\phi(x, y)$ 。
 - 收集人类偏好数据: 对同一个 prompt x , 模型生成两个回答 y_w (win) 和 y_l (lose)。
 - 损失函数 (Pairwise Ranking Loss):

$$\mathcal{L}_{RM} = -\log \sigma(r_\phi(x, y_w) - r_\phi(x, y_l))$$

3. **PPO (Proximal Policy Optimization)**: 利用强化学习优化模型 π_θ 。目标函数:

$$\max_{\pi_\theta} \mathbb{E}[r_\phi(x, y) - \beta \log \frac{\pi_\theta(y|x)}{\pi_{SFT}(y|x)}]$$

注: KL 散度项 $\beta \log(\dots)$ 用于约束模型不要偏离 SFT 模型太远, 防止“奖励模型欺骗 (Reward Hacking)”。

7.4.3 4.3 直接偏好优化 (DPO, Direct Preference Optimization)

核心突破：无需训练奖励模型，无需复杂的强化学习 (PPO)。**推导：**从 RLHF 的最优解公式出发，反解出奖励函数 r 与策略 π 的关系，代入偏好损失。**最终损失函数：**

$$\mathcal{L}_{DPO} = -\mathbb{E}_{(x, y_w, y_l)} \left[\log \sigma \left(\beta \log \frac{\pi_{\theta}(y_w|x)}{\pi_{ref}(y_w|x)} - \beta \log \frac{\pi_{\theta}(y_l|x)}{\pi_{ref}(y_l|x)} \right) \right] \quad (19)$$

直观理解：增加好回答 y_w 的相对概率，降低坏回答 y_l 的相对概率。

7.5 5. 提示学习

7.5.1 5.1 上下文学习 (In-Context Learning, ICL)

无需更新参数，仅通过在 Prompt 中提供示例 (Demonstrations) 让模型学会任务。

- **Zero-shot：**仅提供任务描述。
- **One-shot / Few-shot：**提供 1 个或 K 个示例。

$$\text{Prompt} = I, x_1, y_1, \dots, x_k, y_k, x_{test} \rightarrow \text{LLM} \rightarrow y_{test}$$

7.5.2 5.2 思维链 (Chain-of-Thought, CoT)

核心：Let's think step by step. 在 Prompt 中加入推理步骤 (Rationale)，显著提升模型解决数学、逻辑推理等复杂任务的能力。

- **原理：**将复杂问题 $P(y|x)$ 分解为多步推理 $P(z_1|x)P(z_2|x, z_1) \dots$ 。
- **Zero-shot CoT：**只需追加 "Let's think step by step" 即可激发推理能力。
- **涌现能力 (Emergent Ability)：**CoT 能力通常在模型参数 $> 100B$ 时才显著出现。

8 大模型其他知识 (6 学时张家俊)

复习重点: 本章概览

本章聚焦于 LLM 能力的横向扩展与纵向增强:

1. **多语言大模型**: 解决语言资源不平衡问题, 从英语扩展到多语言。
2. **多模态大模型 (LMM)**: 打破模态壁垒, 实现图/文/音/视频的统一理解与生成。
3. **检索增强生成 (RAG)**: 解决幻觉与知识时效性问题, 是企业级应用落地的首选方案。

8.1 1. 多语言大模型 (Multilingual LLM)

8.1.1 1.1 背景与挑战

- **资源不平衡**: 绝大多数 LLM (如 LLaMA, GPT-3) 以英语为主, 低资源语言表现差。
- **词表扩展**: 原模型词表对非英语支持差 (例如一个汉字被拆分为多个 Token, 效率低)。

8.1.2 1.2 关键技术

- **词表扩充 (Vocabulary Expansion)**: 在原有词表基础上增加目标语言的 Token。需重新训练 Embedding 层, 保持 Transformer 主体参数不变或微调。
- **跨语言指令微调 (Cross-lingual Instruction Tuning)**: 利用翻译数据或多语言指令集 (如 xP3, FLORES), 激发模型的跨语言泛化能力。

8.2 2. 多模态大模型 (Multimodal LLM)

8.2.1 2.1 架构范式

核心思想: 将视觉信号“翻译”为语言模型能理解的 **Embedding**。通用架构 = 视觉编码器 + 连接器 + LLM 基座。

1. **视觉编码器 (Visual Encoder)**: 通常使用 **CLIP (ViT)** 或 **Eva-CLIP**。将图像划分为 Patch, 编码为视觉特征向量。
2. **连接器 (Connector/Projector)**:
 - **Linear Layer (LLaVA)**: 简单的全连接层, 将视觉维度映射到文本维度。
 - **Q-Former (BLIP-2)**: 使用一组可学习的 Query 向量, 从冻结的视觉编码器中提取最相关的视觉特征。
 - **Cross-Attention (Flamingo)**: 在 LLM 层间插入交叉注意力层, 注入视觉信息。
3. **LLM 基座**: 负责推理和生成。通常冻结参数或仅进行 LoRA 微调。

8.2.2 2.2 经典模型

- **CLIP (OpenAI)**: 通过 4 亿图文对进行对比学习 (Contrastive Learning), 对齐了图像和文本空间, 是大多数多模态模型的基石。
- **LLaVA (Large Language and Vision Assistant)**:
 - 创新点: 使用 GPT-4 生成高质量的图文指令数据 (Visual Instruction Tuning)。
 - 训练: 阶段 1-特征对齐 (预训练连接器); 阶段 2-端到端微调。
- **BLIP-2**: 引入 Q-Former, 实现了轻量级的视觉-语言对齐, 大幅降低了训练成本。
- **GPT-4V / Gemini**: 原生多模态 (Native Multimodal), 从预训练阶段就开始混合图文数据。

8.3 3. 检索增强生成 (RAG)

8.3.1 3.1 核心动机

解决 LLM 的两大痛点:

- **幻觉 (Hallucination)**: 一本正经地胡说八道。
- **知识过时 (Knowledge Cutoff)**: 无法回答训练数据截止之后的问题 (如 “昨天那场比赛谁赢了?”)。

8.3.2 3.2 RAG 进化范式

1. **Naive RAG (初级 RAG)**: *Retrieve-Read* 模式。

$$P(y|x) \approx P(y|x, \text{Retrieved Docs})$$

流程: 索引 → 检索 → 生成。缺陷是检索精度低导致生成质量差。

2. **Advanced RAG (高级 RAG)**: 增加了检索前优化 (Query Rewriting) 和检索后优化 (Re-ranking, Summarization)。
3. **Modular RAG (模块化 RAG)**: 将检索、生成、评估拆分为独立模块, 支持迭代检索 (Iterative Retrieval) 和自适应检索 (Adaptive Retrieval, 如 Self-RAG)。

8.3.3 3.3 关键技术细节

A. 索引优化 (Indexing)

- **分块 (Chunking)**:
 - 滑动窗口: 重叠切分, 保持语义连续性。
 - **Small-to-Big**: 检索时匹配小块 (句子级), 生成时送入对应的大块 (窗口或整段), 兼顾检索精度与上下文完整性。
- **元数据过滤**: 在 Chunk 中添加 Year, Author, Title 等元数据, 支持精确过滤。

B. 检索优化 (Retrieval)

- 混合检索 (Hybrid Search):

$$\text{Score} = \alpha \cdot \text{Sparse}(BM25) + (1 - \alpha) \cdot \text{Dense}(\text{Vector})$$

结合关键词匹配（精确匹配专有名词）和语义检索（理解意图）的优势。

- 重排序 (Re-ranking): 使用 Cross-Encoder 对初排回来的 Top-50 文档进行精细打分，取 Top-5 给 LLM。精度提升显著。

C. 生成优化 (Generation)

- Self-RAG: 模型学会输出 [Retrieve], [Relevant], [Support] 等特殊 Token，自我反思是否需要检索以及检索内容是否有用。

8.3.4 3.4 评估体系 (RAGAS)

无需人工标注，使用强模型（如 GPT-4）作为裁判 (LLM-as-a-Judge)。

- 忠实度 (Faithfulness): 答案是否由检索内容推导出来？（检测幻觉）
- 答案相关性 (Answer Relevance): 答案是否回答了用户的问题？
- 上下文相关性 (Context Relevance): 检索到的文档是否包含回答问题所需的信息？

9 NLP 应用任务 (6 学时张家俊)

复习重点: 本章知识图谱

本章聚焦于 NLP 技术在具体场景中的落地应用, 重点掌握两大类任务:

1. 文本分类与聚类: 对整段文本进行理解和归类。
2. 信息抽取 (IE): 从非结构化文本中提取结构化知识 (实体、关系、事件)。

9.1 1. 文本分类 (Text Classification)

9.1.1 1.1 任务定义

给定文本 d 和类别集合 $C = \{c_1, c_2, \dots, c_k\}$, 寻找最优映射 $f: d \rightarrow c_i$ 。典型应用: 垃圾邮件过滤、新闻分类、情感分析。

9.1.2 1.2 传统机器学习方法

核心流程: 特征工程 + 分类器。

- 文本表示: VSM (TF-IDF)。
- 朴素贝叶斯 (Naive Bayes): 基于贝叶斯公式和“特征独立性假设”。

$$P(c|d) = \frac{P(c) \prod P(w_i|c)}{P(d)} \propto P(c) \prod_{i=1}^n P(w_i|c)$$

优点: 训练快, 对短文本效果好; 缺点: 独立性假设过于强, 忽略了词序。

- 支持向量机 (SVM): 寻找一个超平面, 最大化不同类别样本间的间隔 (Margin)。对于非线性可分数据, 利用核函数 (Kernel Trick) 映射到高维空间。

9.1.3 1.3 深度学习方法

- TextCNN: 利用不同宽度的卷积核 (Convolution Filter) 提取 N-gram 特征, 通过 Max-Pooling 捕获最显著特征。
- RNN/LSTM: 利用序列结构捕获长距离依赖。
- Attention/BERT: 利用自注意力机制捕获全局语义, 是目前的主流方法。

9.2 2. 文本聚类 (Text Clustering)

任务: 无监督地将文本集合划分为若干个簇 (Cluster), 使得簇内相似度高, 簇间相似度低。

- K-means: 迭代算法。1. 随机选择 K 个质心; 2. 将样本分配给最近的质心; 3. 更新质心位置。
- 层次聚类 (HAC): 自底向上 (合并) 或自顶向下 (分裂)。不需要预先指定 K 值。

9.3 3. 信息抽取 (Information Extraction)

9.3.1 3.1 命名实体识别 (NER)

任务: 识别文本中的人名 (PER)、地名 (LOC)、机构名 (ORG) 等七类实体。**形式化:** 序列标注问题 (Sequence Labeling)。

- **标注体系:** BIO (Begin, Inside, Outside) 或 BIOES。
- **方法演进:**
 1. **HMM (隐马尔可夫):** 生成式模型, 建模联合概率 $P(X, Y)$ 。
 2. **CRF (条件随机场):** 判别式模型, 建模条件概率 $P(Y|X)$, 解决了 HMM 的独立性假设限制, 是传统方法的 SOTA。
 3. **BiLSTM+CRF:** 深度学习时代的标配。BiLSTM 提取上下文特征, CRF 约束标签转移 (如 B-PER 后不能接 I-LOC)。

9.3.2 3.2 关系抽取 (Relation Extraction)

任务: 给定句子和两个实体, 判断它们之间的语义关系 (如 *Spouse*, *Birthplace*)。例: [姚明] 是 [上海] 人。→ *relation: Citizen_of*

- **有监督方法:** 转化为分类问题。特征包括词汇、词性、句法树路径等。
- **远程监督 (Distant Supervision):** 利用知识库 (如 Freebase) 自动回标数据。假设: 如果在 KB 中 $\langle e_1, e_2 \rangle$ 有关系 r , 则包含这两实体的句子都表达了关系 r 。问题: 带来大量噪声数据 (噪音标签)。
- **联合抽取 (Joint Extraction):** 将实体识别和关系抽取在一个模型中完成, 避免流水线方法的误差传播。

9.3.3 3.3 事件抽取 (Event Extraction)

任务: 从文本中识别特定类型的事件及其论元 (Argument)。例: [9 月 3 日], [习近平] 在 [厦门] 会见 [普京]。

- **触发词 (Trigger):** 会见。
- **事件类型:** Contact.Meet。
- **论元:** 习近平 (Entity), 普京 (Entity), 厦门 (Place), 9 月 3 日 (Time)。

难点: 触发词歧义、论元缺省、多事件共现。

9.4 4. 机器翻译 (Machine Translation)

复习重点:

(注: PPT 目录提及但内容主要集中在分类和 IE, 此处补充核心考点)

- **统计机器翻译 (SMT)**: 基于噪声信道模型 $P(E|F) \propto P(F|E)P(E)$ 。包含翻译模型 (对齐) 和语言模型。
- **神经机器翻译 (NMT)**: 基于 Encoder-Decoder 框架。

$$P(y_t|y_{<t}, x) = \text{softmax}(g(h_t, c_t))$$

其中 c_t 是通过 Attention 机制计算的上下文向量。

- **评价指标**: BLEU (BiLingual Evaluation Understudy), 计算 N-gram 的精确率。

10 课程总结与展望 (3 学时宗成庆)

10.1 课程内容回顾

10.2 学科现状分析

10.3 未来展望

10.4 关于课程考核

11 考核 (2 学时宗成庆)

11.1 考试

附录：常用 LaTeX 模板

1. 学术三线表模板

表 2: 不同模型在测试集上的性能对比（示例）

模型	准确率 (Accuracy)	精确率 (Precision)	召回率 (Recall)	F1 值
SVM	85.2%	84.1%	83.5%	83.8%
Bi-LSTM	89.5%	88.2%	89.0%	88.6%
BERT-Base	92.3%	91.8%	92.1%	91.9%

2. 图片插入模板 (已注释)

3. NLP/模式识别常用数学公式模板

常用符号：数据集 $D = \{(x_1, y_1), \dots, (x_N, y_N)\}$, 损失函数 $\mathcal{L}(\theta)$, 参数 θ 。

贝叶斯公式 (Naive Bayes):

$$P(c|x) = \frac{P(x|c)P(c)}{P(x)} \propto P(c) \prod_{i=1}^d P(x_i|c)$$

Softmax 函数：适用于多分类输出层：

$$\text{Softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}$$

自注意力机制 (Self-Attention): Transformer 的核心公式：

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

交叉熵损失 (Cross Entropy Loss):

$$\mathcal{L} = -\sum_{i=1}^N y_i \log(\hat{y}_i)$$

HMM 前向算法递推:

$$\alpha_t(i) = P(o_1, \dots, o_t, q_t = i | \lambda) = \left[\sum_{j=1}^N \alpha_{t-1}(j) a_{ji} \right] b_i(o_t)$$